Arguments and Debates as Classically Typed Programs

Max RAPP [a]
[a] *FAU Erlangen-Nuremberg*

Abstract. We present a representation of arguments and debates as expressions of the $\bar{\lambda}\mu\tilde{\mu}$-calculus, yielding first-class argument and debate objects with computational content: successful arguments return normally, defeated ones throw an exception. Arguments and counterarguments are constructed in the linear, intuitionistic fragment of the calculus. They can then be composed into debate expressions with non-linear control flow. We show how such a representation handles conflicting terms, control-flow manipulation as well as attack and support between arguments. The encoding makes use of throw, catch and steal constructions that mirror exception handling. Support patterns can be evaluated by discarding redundant alternatives. Attack patterns behave as conditional aborts that throw an exception if the attacked argument is defeated. The resulting AC/DC sublanguage is classical which is shown via a debate term reducing to a proof of Peirce's law.

Keywords. argumentation, proof terms, classical logic, debate

1. Introduction

Argumentation Frameworks such as ASPIC+ [1] [2], ABA [3], Carneades [4], Deductive Argumentation [5] or Sequent-based Argumentation [6] and their implementations provide argumentative superstructure for a given object logic and enable non-monotonic inference through the (automated) construction and evaluation of arguments and their relations. However, they are less focussed on the arguments themselves as first-class objects. Logical Frameworks such as LF [7], COC [8] and Higher-Order Logic [9] and their implementations such as MMT [10], ROCQ [11], Lean [12], and Isabelle/HOL [13] use variants of the propositions-as-types paradigm to enable the mechanization of logics. In particular, mechanizing a logic's proof theory yields first-class proof objects that can be used to provide further services (e.g. interactive assisted proof construction and rewriting), reused in other proofs or managed in libraries for scalable development. However, Logical Frameworks are not well-suited to shallowly embed non-monotonic formalisms such as argumentation frameworks as their foundations are fundamentally monotonic. While deep embeddings are possible [14] they do not enjoy the same degree of tool support. Hence, to the best of our knowledge, no existing system combines the argument-construction and evaluation mechanisms of structured argumentation with the first-class proof objects and generic tooling of logical frameworks. In this paper we take a step in this direction by providing an arguments-as-programs view on argumentation through a representation of arguments and debates as expressions of classical type theory.

The outline of the paper is as follows: in Section 2 we motivate our approach while recapping related work in this area. In Section 3 we introduce classical type theory in the form of a dialectical variant of the $\bar{\lambda}\mu\tilde{\mu}$-calculus, typing its (control) terms and contexts by (classical) proofs and refutations. We likewise introduce constructs for throwing, stealing and catching of subexpressions. Section 4 introduces arguments as open $\bar{\lambda}\mu\tilde{\mu}$-terms and the support and attack constructs to compose debates. The full debate language is presented in Section 5 where we also show that it is complete with respect to the original, classical calculus. Finally we conclude in Section 6 outlining directions for future work.

2. Arguments and Debates in the Propositions-as-Types Paradigm

Famously one can understand the proofs of intuitionistic logic for a given proposition as the programs of a corresponding type (most canonically for propositional logic and the simply-typed lambda calculus) It is hence natural to ask if the correspondence can be extended to the *arguments* for a given proposition. Indeed such an approach was pursued early on in the development of the field of formal argumentation by Krause et al. 1995 [15],[16]. Krause et al. represent arguments as (open) terms of the minimal simply-typed λ-calculus. However, this initial work failed to gain many followers (an exception is the work by Pandžić 2022 [17], who presents a defeasible justification logic in which argument terms feature in modalities): argumentation poses particular difficulties due to conflicting terms, control flow manipulations and the need to compile debate expressions. We illustrate these challenges by way of an example loosely based on the Lorenzen game for sequent calculus (see chapter 7 of Sørensen and Urzyczyn 2006 [18]).

Example 1 (Running Example: A Prover-Skeptic-Dialogue)**.** Consider two logicians arguing about the truth of Peirce's law:

P_1: I assert $((P \to Q) \to P) \to P$!

S_1: I don't believe you! Assume $(P \to Q) \to P$! Can you really prove P?

P_2: I could take your word and prove P by asserting $P \to Q$. But I don't believe *you*! Assume $P \to Q$. Can you show P?

S_2: Why should I take your word for $P \to Q$. Let me grant you P! Can you prove Q?

P_3: Ha! I trapped you. You have granted me P and that is exactly what you asked me to prove earlier. So you both affirm and deny P. Hence I prove Q by contradiction.

S_3: Alright, I retract my offer of P. Instead I will take your offer $P \to Q$ and prove P. I do so by asserting $(P \to Q) \to P$. By $(P \to Q) \to P$ and $P \to Q$, P.

P: Ha! Got you again: now you proved P which you deny. Contradiction!

S: Alright, I am out of options, you win.

Conflicts: Prover asserts $((P \to Q) \to P) \to P$ but Skeptic denies this. Following Krause et al's approach, we could represent the arguments as terms $p : ((P \to Q) \to P) \to P$ and $s : \neg((P \to Q) \to P) \to P$, respectively. But how are we to construct a term $d = ?(p,s) : ((P \to Q) \to P) \to P$ that suitably combines argument and counterargument into a debate about $((P \to Q) \to P) \to P$?

Control Flow: Throughout our example Prover and Skeptic challenge each other to prove subgoals necessary for the continuation of the debate. However, each side can choose which claim it argues at any given time: in this way, Prover sets a trap for Skeptic goading her into assuming P. Then she changes the topic back to an earlier challenge and points out that Skeptic has contradicted herself. Such topic changes and argumentative maneuvering are par for the course in *concrete* argumentation. If we wish to represent them we need a way to manipulate the *control flow* of debate terms.

Compilation: Prover eventually ends the debate by closing all of Skeptic's possible response avenues (corresponding to a classical proof). In general, however, the use of implicit assumptions or defeasible defaults in arguments means that a given debate can always continue or - conversely - is never incomplete. Hence we should be able to represent the debate between Prover and Skeptic at different times and obtain debate terms $d_1, \ldots, d_n$ that are built incrementally from each other. That is, we need compilation operations that allow us to construct debate terms from the arguments and counterarguments currently in the context.

In this paper we focus on the former two aspects, leaving (interactive) compilation mostly for future work. Handling the phenomena of conflict and control flow requires a richer type theory that can handle and represent both individual arguments and debates in a unified language.

3. Classical Type Theory, Dialectics and Control

Griffin, Parigot, Wadler [19,20,21] and Curien and Herbelin [22] extend the proposition-as-types correspondence to classical logic. Griffin showed that *control operators* such as call-cc can be typed by Peirce's law; Parigot's $\lambda\mu$-calculus provides the Curry-Howard-Correspondent to classical natural deduction; Wadler as well as Curien and Herbelin invented symmetric systems, in the latter case the $\bar{\lambda}\mu\tilde{\mu}$ calculus which is to the classical sequent calculus LK as $\lambda\mu$-calculus is to classical natural deduction.

The latter system can be transposed into a dialectical natural deduction system [23, 24,25] of *proofs and refutations*, providing us with the second one of our requirements.

3.1. The $\bar{\lambda}\mu\tilde{\mu}$ Calculus - Dialectical Variant

We recap the $\bar{\lambda}\mu\tilde{\mu}$ Calculus and give a dialectical variant of its syntax.

Expressions are divided into terms, contexts and commands. Terms are read left-to-right while contexts are read right-to-left. Commands are read either left-to-right or right-to-left depending on how they are bound. The terminology on the expression level gestures at an interpretation of $\bar{\lambda}\mu\tilde{\mu}$-expressions as program terms and continuation contexts. Terms are expressions with outer holes that wait to be passed to a context. Contexts are expressions with inner holes that await a term to be passed to them. Commands assign terms to contexts. The μ binder when binding to the right represents a call-with-current-continuation. Symmetrically, when binding to the left μ corresponds to a call-with-current-term.

We say that a μ-bound variable is *affine* if it does not appear free in the command in which it is bound (likewise we call a binder binding an affine variable an *affine binder*). If

$$
\begin{array}{llll}
v & ::= & x \mid \lambda x:\sigma.v \mid E \circ v \mid \mu\sigma:\alpha.c & \textit{Terms} \\
E & ::= & \alpha \mid E.\sigma:\alpha\lambda \mid v \circ E \mid c.x:\sigma\mu & \textit{Contexts} \\
c & ::= & \rangle v:\sigma:E\langle & \textit{Commands} \\
\sigma,\delta & ::= & a \mid \sigma \to \delta \mid \sigma - \delta & \textit{Propositions}
\end{array}
$$

Grammar 1 Grammar of the $\bar{\lambda}\mu\tilde{\mu}$-calculus

we want to explicitly indicate that a binder is affine we write $\mu\_.c$ and $c.\_\mu$, respectively. To denote that t' is a subexpression of t, we write $t[t']$. For variables we take $t[x], t[\alpha]$ to mean that x, α occur free in t. We call such expressions *open expressions*. Finally, we write $t'[x \leftarrow t]$ for the usual capture-avoiding substitution while $t'[t/x]$ denotes non-capture-avoiding, non-recursive graftings. Two fragments of the syntax are of interest: *Intuitionistic Logic* is obtained by restricting the domain of the context-variables α to at most a singleton set. Conversely, *Dual-intuitionistic Logic* is obtained by restricting the term variable domain to at most a singleton. The intuitionistic and dual-intuitionistic fragments can alternatively be obtained by restricting expressions to *linear* terms and contexts:

Definition 1 (Linear Expressions)**.** A variable x is *linearly bound* in an expression t with outer binder b iff x is bound by b in t, appears exactly once in t and between b and x there is no binder occurrence of the same kind as b. A $\bar{\lambda}\mu\tilde{\mu}$ expression is called *linear* iff all context variables bound in it are bound linearly. It is called *dual-linear* if all term variables bound in it are bound linearly.

The intuitionistic fragment corresponds to the sublanguage of linear $\bar{\lambda}\mu\tilde{\mu}$ expressions, the dual-intuitionistic fragment to the sublanguage of dual-linear $\bar{\lambda}\mu\tilde{\mu}$ expressions.

Judgments and Typing: Sequents have the form *Antecedent* $\vdash$ *Stoup* $\dashv$ *Succedent*. Antecedent and Succedent are conjunctive contexts of terms and contexts respectively while the stoup contains exactly one active formula - a term, context or command - at any given time. They are read in the same direction as their elements. On the type-level, we give a dialectical interpretation based on [23], [24] and [25].

There are three judgments:

1. $\Sigma \vdash v:\sigma \dashv \Delta$ judges v to be a *proof* for σ
2. $\Sigma \vdash \sigma:E \dashv \Delta$ judges E to be a *refutation* of σ
3. $\Sigma \vdash \rangle v:\sigma:E\langle \dashv \Delta$ judges v and E to form a *contradiction* at σ

Terms correspond to proofs, contexts to refutations and commands to contradictions. Computationally, propositions type terms that produce a witness for their truth regardless of input. Conversely they type contexts that halt computation upon being passed a proof witness for their proposition, allowing for arbitrary returns. It follows that commands can return arbitrary outputs from arbitrary inputs. The proofs and refutations of a given proposition are called *contraries*.

Curien and Herbelin [22] type commands by a sequent with an empty stoup mirroring the usual cut rule. However, we want to keep track of the issue of the contradiction and hence use the following typing rule:

$$\frac{\Sigma \vdash v : \sigma \dashv \Delta \qquad \Sigma \vdash \sigma : E \dashv \Delta}{\Sigma \vdash \rangle v : \sigma : E \langle \dashv \Delta}$$

Instead of cutting the issue, the meeting of a contrary proof and refutation yields a contradiction. Proofs have the following typing rules (the $\perp$-symbol is a meta-variable ranging over propositions):

$$\frac{}{\Sigma, x : \sigma \vdash x : \sigma \dashv \Delta} \qquad \frac{\Sigma, x : \sigma \vdash v : \delta \dashv \Delta}{\Sigma \vdash \lambda x : \sigma . v : \sigma \to \delta \dashv \Delta}$$

$$\frac{\Sigma \vdash v : \delta \dashv \Delta \qquad \Sigma \vdash \sigma : E \dashv \Delta}{\Sigma \vdash v \circ E : \delta - \sigma \dashv \Delta} \qquad \frac{\Sigma \vdash c : \perp \dashv \Delta, \delta : \alpha}{\Sigma \vdash \mu \alpha . c : \delta \dashv \Delta}$$

These are mirrored by the typing rules for refutations:

$$\frac{}{\Sigma \vdash \delta : \alpha \dashv \delta : \alpha, \Delta} \qquad \frac{\Sigma \vdash \delta : F \dashv \sigma : \alpha, \Delta}{\Sigma \vdash \delta - \sigma : F . \alpha \lambda \dashv \Delta}$$

$$\frac{\Sigma \vdash v : \sigma \dashv \Delta \qquad \Sigma \vdash \delta : E \dashv \Delta}{\Sigma \vdash \sigma \to \delta : v \circ E : \dashv \Delta} \qquad \frac{\Sigma, x : \sigma \vdash c : \perp \dashv \Delta}{\Sigma \vdash \sigma : c . x \mu \dashv \Delta}$$

Example 2 (Implication Elimination in $\bar{\lambda}\mu\tilde{\mu}$)**.** We can recover elimination forms as derived rules. In particular implication elimination (application) is given by the term $\lambda x : \sigma . \lambda y : \sigma \to \delta . \mu \alpha : \delta . \rangle y : \sigma \to \delta : x \circ \alpha \langle$ and typed as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma, f : \sigma \to \delta \vdash f \dashv \Delta} \quad \dfrac{\dfrac{}{\Gamma, x : \sigma \vdash x \dashv \Delta} \quad \dfrac{}{\Gamma \vdash \alpha \dashv \delta : \alpha, \Delta}}{\Gamma, x : \sigma \vdash \sigma \to \delta : x \circ \alpha \dashv \delta : \alpha, \Delta}}{\Gamma, x : \sigma, f : \sigma \to \delta \vdash \rangle f :: x \circ \alpha \langle \dashv \delta : \alpha, \Delta}}{\Gamma, x : \sigma, f : \sigma \to \delta \vdash \mu \alpha : \delta . \rangle f :: x \circ \alpha \langle : \delta \dashv \Delta}}{\Gamma, x : \sigma \vdash \lambda f : \sigma \to \delta . \mu \alpha : \delta . \rangle f :: x \circ \alpha \langle : (\sigma \to \delta) \to \delta \dashv \Delta}}{\Gamma \vdash \lambda x : \sigma . \lambda f : \sigma \to \delta . \mu \alpha : \delta . \rangle f :: x \circ \alpha \langle : \sigma \to (\sigma \to \delta) \to \delta \dashv \Delta}$$

Note that this is a minimal term which shows that the standard ND-calculi for minimal and intuitionistic calculus can be recovered.

Finally, the calculus' expressions can be reduced as follows:

$$\begin{array}{rrcr}
(\to) & \rangle \lambda x . v_1 :: v_2 \circ E \langle & \to & \rangle v_2 :: \rangle v_1 :: E \langle . x \mu \langle \\
(-) & \rangle E_1 \circ v :: E_2 . \alpha \lambda \langle & \to & \rangle \mu \alpha . \rangle v :: E_2 \langle :: E_1 \langle \\
(\mu <) & \rangle \mu \alpha . c :: E \langle & \to & c[\alpha \leftarrow E] \\
(> \mu) & \rangle v :: c . x \mu \langle & \to & c[x \leftarrow v]
\end{array}$$

Commands of the shape $\rangle \mu \alpha . v :: E . x \mu \langle$ form *critical pairs*: their reduction is non-deterministic. *Evaluation strategies* determine the order of reduction in such cases. Two

canonical strategies are call-by-value evaluation which is obtained by applying $(\mu <)$-reductions first and call-by-name which applies $(> \mu)$-reductions first.

Additionally, we have the following η-equivalence rules for μ binders:

$$\begin{array}{lcl} (\mu. - \eta) & v & \leftrightarrow \quad \mu\alpha.\rangle v :: \alpha\langle \\ (.\mu - \eta) & E & \leftrightarrow \quad \rangle x :: E\langle .x\mu \end{array}$$

3.2. Control Flow: Affine Binders, Throw, Steal and Catch

The reduction system brings to the fore how control flow is modeled in the $\bar{\lambda}\mu\tilde{\mu}$-calculus. Of particular interest to us is the reduction behavior of affine terms. Note that affine μ binders throw away their contexts (terms) during reduction:

$$\mu\beta.\rangle\mu\_.\rangle v :: E\langle :: F[\beta]\langle \longrightarrow_{\mu<} \mu\beta.\rangle v :: E\langle$$

Further, so long as β does not appear free in v, E, the outer $\mu\beta$ becomes affine itself. In this way the inner command "bubbles up" mirroring the behavior of *exceptions* in common programming languages. For this reason, classical type theory has long been used as a setting to study exception handling (see [26]) for an overview. However, the target calculus is usually Parigot's $\lambda\mu$-calculus . We adopt some of the notions developed in that context to the $\bar{\lambda}\mu\tilde{\mu}$-calculus. For present purposes, only the following minimal account is necessary, a full treatment is left to future work. We define the following subgrammar of affine expressions :

$$\begin{array}{llcl} \text{throw} & ::= & \mu\sigma : \_.\rangle v : \sigma : \alpha\langle \quad | \quad \rangle x : \sigma : E\langle .\_ : \sigma\mu & \textit{Throw Terms} \\ \text{steal} & ::= & \mu\sigma : \_.\rangle x : \sigma : E\langle \quad | \quad \rangle v : \sigma : \alpha\langle .\_ : \sigma\mu & \textit{Steal Terms} \end{array}$$

Grammar 2 Throw and Steal

Notation 1 (Throw/Steal)**.** Throw and steal terms are abbreviated as $throw(v, : \alpha)$ and $(x :, E)steal$, respectively; throw and steal contexts as $(x :, E)throw$ and $steal(v, : \alpha)$, respectively.

Throw and steal are purely bureaucratic operations: they preserve typing. In the throw case, the current context is discarded and the current term v (context E) is thrown to an outer catch context (term). The pathological command will continue to bubble up until it gets *caught* by a binder that binds some variable free in the exception command c. This mirrors the computational behavior of exception handling, where v (E) is the pathological term (context) to be handled. However, unlike in the usual treatment of exceptions - the exception v is not thrown to some particular catch context but can in principle be catchable on (any number of) term or context variables occurring free in it.

Thus any μ-abstracted term or context binding a variable free in the thrown command can serve as a catch.

Notation 2 (Catch)**.** We write $catch(: \alpha)$ and $(x :)catch$, for term (context) side μ binders, respectively, if we wish to stress their role as a catch statement.

In the steal case, the current term (context) is discarded and an inner term v (context E) is returned to the current context α (term x). This behavior can be thought of as an access-restricted computation that checks the context before returning to it and if it does not have access rights to the current value, returns another value instead.

Steal terms allow the modeling of total failure or abortion of a computation, as the following example demonstrates:

Example 3 (Contradictory Proof terms)**.** Assume atoms $t : A$ and $A : t'$. Consider the following proof term:

$$\mu A : \alpha.\rangle t ::\rangle \mu\_.\rangle x :: t' \langle :: \alpha \langle .x : A \mu \langle : A$$

Given that we have a contradiction, simply returning t to α would hide the pathology. Instead, the contradiction should be brought to the surface. The context $\rangle \mu\_.\rangle x :: t' \langle :: \alpha \langle .x : A \mu$ thus denies t access to the outer context α and instead reroutes it to the abort context t'. The term hence reduces to

$$\mu A : \alpha.\rangle t : A : t' \langle,$$

bringing the contradiction to the top and yielding an uncatchable exception that will consume any context to which it is passed - in other words *aborting* the computation.

4. Modeling Arguments and their Relations in the dialectical $\bar{\lambda}\mu\tilde{\mu}$-calculus

We generalize the approach taken by Krause et al. [15] to define arguments in the $\bar{\lambda}\mu\tilde{\mu}$-setting. The basic idea is that a successful argument should correspond to a program that returns normally, while a defeated argument should abort the computation.

Definition 2 (Arguments)**.** We call any linear term $t : \sigma$ an *argument for* σ. A dual-linear context $\sigma : t$ is called an *argument against* σ or *counterargument to* σ. An *argument* is either an argument for or against some proposition σ. Arguments are not required to be closed expressions: the free variables in an argument t are called the *assumptions* of t.

Two design choices require justification: firstly, why are (counter-)arguments restricted to the (dual-)intuitionistic fragment? Secondly, why use free variables as an argument's assumptions?

Arguments and Control Flow: The motivation for the (dual-)intuitionistic restriction is twofold: from a logical perspective the divide between constructive intuitionistic proofs and non-constructive classical proofs - i.e. demonstrations, that the given proposition cannot be refuted - map congenially to a corresponding dialectical distinction between concrete arguments for (against) a given proposition and debates about a proposition where the proponent (opponent) has a winning strategy.Computationally speaking, the move to the classical calculus introduces precisely the non-linearity (e.g. traps or topic changes) that distinguishes single-agent, sequential reasoning from multi-agent, adversarial reasoning.

Assumptions: There are three broad approaches in the literature: assumptions as meta-level defeasible premises or inference rules (e.g. [3],); assumptions as object logic formulae or axioms (e.g. [5]) and assumptions as structural components of proof calculi, e.g in sequent antecedents (e.g.[6]). While we certainly regard a dedicated treatment of assumptions as desirable we leave this to future work and stay close to the latter approach - if only for its simplicity. However, our notion of arguments requires that assumptions appear in the concrete expressions instead of only in the abstract sequents. This has the advantage that attacks/supports can only target assumptions that are actually used by the targeted argument.

Example 4 (Prover-Skeptic-Dialogue: Arguments)**.** We formalize the arguments from Example 1, using the same free variables across arguments:

$$P_1 : x \vdash \mu((P \to Q) \to P) \to P : \alpha.\rangle x :: \alpha\langle \dashv \tag{1}$$

$$S_1 : x, y \vdash \mu((P \to Q) \to P) \to P : \beta.\rangle \lambda f : ((P \to Q) \to P).y :: \beta\langle \dashv \tag{2}$$

$$P_2 : x, y, z, r \vdash \mu P : \gamma.\rangle \lambda g.z : ((P \to Q) \to P) : r \circ \gamma\langle \dashv \tag{3}$$

$$S_2 : x, y, z, r, u \vdash \mu P : \varepsilon.\rangle \lambda h.u : P \to Q : \zeta\langle \dashv \zeta \tag{4}$$

$$P_3 : x, y, z, r, u \vdash \mu Q : \theta.\rangle h : P : \gamma\langle \dashv \zeta \tag{5}$$

$$S_3 : x, y, z, r, u, s \vdash \rangle f : (P \to Q) \to P : s \circ \iota\langle .s : P \to Q\mu \dashv \zeta, \iota \tag{6}$$

$$P_4 : x, y, z, r, u, s \vdash \rangle w :: \gamma\langle .w : P\mu \dashv \zeta, \iota \tag{7}$$

Assertions made by the opponents are represented by free variables, offers by bound ones. Generally the onus to prove an assertion rests with the person who claims it, but the onus can be passed to the opposing side by making an offer (i.e. the assertion is the bound term in a λ-term ($-$ context). For example, in S_1, the subterm $\lambda f : ((P \to Q) \to P).y$ represents the challenge to Prover to derive P given $((P \to Q) \to P)$. The μ-binders are used to respond to a challenge, bind it and redirect control-flow to a lemma or contradiction one wants to prove to meet the challenge. In P_2 Prover decides to work on the lemma $(P \to Q) \to P$. She could take prover's offer f to prove the lemma and proceed to derive its antecedent $r : (P \to Q)$ to prove the challenged proposition P from the lemma. However, in this case she instead decides to challenge Skeptic back on his offer, laying a trap. In S_2 and Skeptic walks into the trap by offering $h : P$ (even though she could have tried to prove P from $P \to Q$ (ζ). Prover exploits this in P_3 where she uses the offered variable h to meet Skeptic's earlier challenge γ. Skeptic then backtracks and instead tries to proceed with her only open alternative ζ from S_2, but ultimately to no avail.

4.1. Argument Relations

In this section we will put the throw, steal and catch patterns we developed in Section 3.2 to work to model the argumentative relations of attack (rebut and undermine) and support (buttress and undergird).

Support: The basic idea is that we can use throw-steal-catch patterns to model alternative arguments for a proposition. This approach follows Autexier's and Sacerdoti-Coen's approach [27], who develop a pattern that allows the encoding of alternative proofs in $\bar{\lambda}\mu\tilde{\mu}$-terms:

$$alt(t_1,t_2) : \phi = \mu\alpha.\rangle\mu\_.\rangle t_1 : \phi : \alpha\langle::\rangle t_2 :: \alpha\langle.\_\mu\langle$$

Using the concepts developed in Section 3.2, alternative proofs (refutations) can be viewed as the *throw-steal-catch patterns*.

Definition 3 (Support)**.** Support is defined via the following grammar:

$$\begin{array}{rcl} t_1 \prec_\phi t_2 & ::= & \text{catch}(:\alpha).\rangle\,\text{throw}(t_1,:\alpha) : \phi : \text{steal}(t_2,:\alpha)\langle \\ & \mid & \rangle(x:,t_1)\,\text{steal} : \phi : (x:,t_2)\,\text{throw}\langle.(x:)\,\text{catch} \end{array}$$

Grammar 3 Support

An argument t_1 *supports* an argument t_2 iff $t_2 \prec t_1$. We say that t_1 *undergirds* t_2 iff t_2 is a variable. Else t_1 *buttresses* t_2. We define $\prec$ to be right-associative.

Support in our approach is a disjunctive operation: each supporter (unless defeated) constitutes a sufficient condition for the supported conclusion.

Example 5 (Support)**.** Consider the arguments P_2 and S_2 from Example 4. We have $P_2[y : P]$ and $S_2 : P$. Hence $P_2[y \prec_P S_2/y]$ is an expression encoding that P_2 is supported by S_2 on P. It can be evaluated either call-by-name returning y or call-by-value returning S_2.

Attack can be encoded by giving two alternative arguments for the thesis: the original one and one from contradiction. Attack in structured argumentation is often modeled on top of the base logic [3],[6]. We instead define contrariness bottom-up by treating proofs and refutations as contrary to each other.[1]

Definition 4 (Attack)**.** Let t_1,t_2 be intuitionistic $\bar{\lambda}\mu\tilde{\mu}$ expressions. An *attack* between t_1 and t_2 is defined as one of the patterns in Grammar 4.

$$\begin{array}{rcl} t_1 \leftsquigarrow_\phi t_2 & ::= & \text{catch}(:\alpha).\rangle t_1 : \phi :\rangle(x:,t_2)\,\text{steal} :: (x:,\alpha)\,\text{throw}\langle.(x:)\,\text{catch}\langle \\ & \mid & \rangle\,\text{catch}(:\alpha).\rangle\,\text{throw}(x,:\alpha) : \phi : \text{steal}(t_2,:\alpha)\langle:\phi : t_1\langle.(x:)\,\text{catch} \end{array}$$

Grammar 4 Attacks

We say that t_2 attacks t_1 on ϕ iff $t_1 \leftsquigarrow_\phi t_2$. In particular, if t_1 is an assumption, t_2 *undermines* t_1. Else t_2 *rebuts* t_1

[1] Note that in the classical calculus contrariness defined in this way becomes classical negation. But this need not be so: in the minima l and intuitionistic fragment, the equivalence between the two notions does not hold.

Similarly to Example 3, in the case of an attack we have two conflicting terms t_1 and t_2. However, we drop the assumption that t_1, t_2 be atoms. Hence t_1, t_2 may themselves throw to outer contexts or steal values. The crucial difference between attacking arguments and conventional exception handling in programming lanuguages is that the attacker can *itself* be defeated. Hence a counterargument should only be allowed to steal a the return context if its own return context is not stolen. However, we leave these dynamics to future work and will presently only address the simple case of unattacked attackers.

Example 6 (Attack)**.** Consider again the arguments P_1 from Example 4. Skeptic might feel confident and decide to assume the onus by attempting to *refute* Prover's claim instead of challenging Prover to derive it from an assumption. In this case she might put forward an argument $S_1' \langle s :: \chi \rangle . s : ((P \to Q) \to P) \to P\mu$. Then $P_1[x \leftarrow\!\!\!\sim_{((P\to Q)\to P)\to P} S_1'/x]$ is an expression that models this attack. If S_1 is undefeated, it will steal x from its context and instead return an exception $\mu\_.\rangle x :: \chi\langle$ which - unless caught on $\chi or x$ will bubble up all the way to the top context. Otherwise, x is returned normally.

Finally we note that just like throw and steal, support and attack are purely bureaucratic and hence type-preserving.

5. Debates in the $\bar{\lambda}\mu\tilde{\mu}$-calculus: The AC/DC Sublanguage

We now put the ingredients from the previous sections together to define a sublanguage that can represent debates including both support and attacks between arguments (and subdebates).

Definition 5 (AC/DC Debate Terms)**.** Let v_l, E_l range over the linear (dual-linear) $\bar{\lambda}\mu\tilde{\mu}$ expressions. Then AC/DC debate terms are given by the following subgrammar:

$$\mathrm{d} \quad ::= \quad v_l \quad | \quad E_l \quad | \quad d \prec_\sigma d' \quad | \quad d \leftarrow\!\!\!\sim_\sigma d' \quad | \quad d[d'/x]$$

Grammar 5 Debates

Example 7 (Debate Term for Peirce's Law)**.** From the arguments given in the previous examples, we can construct the following debate term:

$$(P_1[x \prec S_1[y \prec (P_2[z \prec S_2[u \prec P_3/u]/z])[\zeta \prec S_3[\iota \prec P_4/\iota]/\zeta]/y]/x]) \leftarrow\!\!\!\sim S_1'.$$

This term expands to $(\mu\alpha.\rangle x \prec_{((P\to Q)\to P)\to P} \mu\beta.\rangle \lambda f.y \prec_P \mu P : \gamma.\rangle \lambda g.z \prec_P \mu P : \varepsilon.\rangle \lambda h.u \prec_Q \mu Q : \theta.\rangle h :: \gamma\langle :: \zeta \prec_{P\to Q}\rangle :: s \circ \iota \prec_P \rangle w :: \gamma\langle .w : P\mu\langle .s : P \to Q\mu\langle :: r \circ \gamma\langle :: \beta\langle :: \alpha\langle\rangle \leftarrow\!\!\!\sim\rangle s :: \chi\langle .s\mu$

5.1. Evaluating Debates: Redundancy and Defeat

In order to flesh out our idea that argument and debate computations should return differently based on their acceptance status, we investigate the evaluation properties of debates. We leave a full analysis of evaluation strategies for AC/DC to future work, here we

only give a partial strategy. When evaluating support patterns, one might decide to keep all supporters or only a subset of strongest supporters, in particular if some supporters are redundant. We define redundancy via strictness:

Definition 6 (Strictness)**.** Let d, d' be debates such that $d[d']$. We say that d' is *strict* in d iff every free variable in d' is bound in d.

Some care is required to account for caught variables: an argument cannot be discarded if it is used for a proof-by-contradiction of the contrary of (i.e. gets caught on) one of its assumptions:

Definition 7 (Strict Redundancy)**.** Let d, d_1, d_2 be debates such that $d_1 \prec_\phi d_2$ is a subexpression of d. Then d_2 is *strictly redundant* with respect to d_1 in d iff d_1 is strict in d and d_2 is either not strict in d or a proof (refutation). Else if d_1 is not strict in d and d_2 is a strict in d, d_1 is strictly redundant with respect to d_2.

In the case of attack patterns, the defeated alternative should be discarded. Like before, we define only a notion of strict defeat:

Definition 8 (Strict Defeat)**.** Let d, d_1, d_2 be debates such that $d_1 \leftsquigarrow_\phi d_2$ is a subexpression of d. Then d_2 *strictly defeats* d_1 iff d_2 is a proof (refutation) and d_1 is either a refufation (proof) or not strict in d. Else if d_1 is a proof (refutation) or strict in d and d_2 is not strict in d, d_1 strictly defeats d_2

Thus even proofs can be strictly defeated in inconsistent theories. Finally, we define the following partial evaluation strategy based on strict defeat and redundancy:

Definition 9 (Call-By-Onus)**.** Given a debate $d_1 \leftsquigarrow_\phi d_2$ with command c, $d_1 : \phi$ ($\phi : d_1$)and $\phi : d_2$ ($d_2 : \phi$), such that d_2 strictly defeats d_1, apply $\mu < (> \mu)$ to c. Else if d_1 strictly defeats d_2, apply $> \mu$ ($\mu <$) to c. Else, if for some $i \in \{1,2\}$ d_i is not normal form, delay evaluation of c and continue by evaluating d_i. Likewise, given a debate $d_1 \prec_\phi d_2$ with command c, $d_1 : \phi$ ($\phi : d_1$)and $\phi : d_2$ ($d_2 : \phi$), such that d_2 is strictly redundant with respect to d_1, apply $\mu < (> \mu)$ to c. Else if d_1 strictly defeats $d_2$apply $> \mu$ ($\mu <$) to c. Else, if for some $i \in \{1,2\}$ d_i is not normal form, delay evaluation of c and continue by evaluating d_i

Example 8 (Normalizing the Debate for Peirce's Law)**.** Consider again the debate $(\mu\alpha.\rangle x \prec_{((P\to Q)\to P)\to P} \mu\beta.\rangle\lambda f.y \prec_P \mu P : \gamma.\rangle\lambda g.z \prec_P \mu P : \varepsilon.\rangle\lambda h.u \prec_Q \mu Q : \theta.\rangle h :: \gamma\langle :: \zeta \prec_{P\to Q}\rangle f :: s \circ \iota \prec_P\rangle w :: \gamma\langle .w : P\mu\langle .s : P \to Q\mu\langle :: r \circ \gamma\langle :: \beta\langle :: \alpha\langle\rangle \leftsquigarrow\rangle s :: \chi\langle .s\mu$. The outer attacker is non-strict, so we have to delay the evaluation of the outer attack pattern. Entering the attacked argument, we have to delay evaluation three times until we can discard u as well as ζ: $\mu(\alpha.\rangle x \prec_{((P\to Q)\to P)\to P} \mu\beta.\rangle\lambda f.y \prec_P \mu P : \gamma.\rangle\lambda g.z \prec_P \mu P : \varepsilon.\rangle\lambda h.\mu Q : \theta.\rangle h :: \gamma\langle ::\rangle f :: s \circ \iota \prec_P\rangle w :: \gamma\langle .w : P\mu\langle .s : P \to Q\mu\langle :: r \circ \gamma\langle :: \beta\langle :: \alpha\langle\rangle \leftsquigarrow\rangle s :: \chi\langle .s\mu$. Next, applying $> \mu$ and discarding ι we get: $(\mu\alpha.\rangle x \prec_{((P\to Q)\to P)\to P} \mu\beta.\rangle\lambda f.y \prec_P \mu P : \gamma.\rangle\lambda g.z \prec_P \mu P : \varepsilon.\rangle f :: \lambda h.\mu Q : \theta.\rangle h :: \gamma\langle\circ\rangle w :: \gamma\langle .w : P\mu\langle :: r \circ \gamma\langle :: \beta\langle :: \alpha\langle\rangle \leftsquigarrow\rangle s :: \chi\langle .s\mu$. Now the subterm $\mu P : \varepsilon.\rangle f :: \lambda h.\mu Q : \theta.\rangle h :: \gamma\langle\circ\rangle w :: \gamma\langle .w : P\mu\langle$ has become strict and with no strict terms left, it will bubble up, finally yielding a term for Peirce's law: $(\mu\alpha.\rangle\lambda f.\mu\gamma.\rangle f :: \lambda h.\rangle h :: \gamma\langle\circ\gamma\langle :: \alpha\langle\rangle \leftsquigarrow\rangle s :: \chi\langle .s\mu$. Since Peirce's law is strict, the outer attacker is strictly defeated and finally gets discarded:

$$\mu\alpha.\rangle\lambda f.\mu\gamma.\rangle f :: \lambda h.\rangle h :: \gamma\langle\circ\gamma\langle :: \alpha\langle : ((P \to Q) \to P) \to P$$

5.2. Classicality of AC/DC

Finally, we shall recover classical expressivity of the $\bar{\lambda}\mu\tilde{\mu}$-calculus by showing how to obtain a proof for every classical tautology by normalizing *AC/DC* debates.

Definition 10 (Reduction-closed Fragment)**.** The *$\mathscr{S}$-reduction-closed* fragment of *AC/DC* is the set of *AC/DC* terms that can be reduced via some evaluation strategy $\mathscr{S}$ to a closed $\bar{\lambda}\mu\tilde{\mu}$ expression.

Proposition 1 (Soundness of *AC/DC*)**.** *The $\mathscr{S}$ reduction-closed fragment of AC/DC is sound with respect to classical propositional logic.*

Proof. By subject reduction and well-typedness of *AC/DC* terms. □

Proposition 2 (Completeness of *AC/DC*)**.** *The reduction-closed fragment of AC/DC is complete with respect to classical propositional logic.*

Proof. This follows from the fact that intuitionistic logic extended with Peirce's law is a complete proof system for classical logic and that all intuitionistic terms are atomic *AC/DC* debates: let T be a propositional tautology. There exists an intuitionistic argument $t_0 : T$ with finitely many free variables $x_1, \ldots, x_n$ such that for all $i \leq n$, $x_i : ((\sigma_i \to \delta_i) \to \sigma_i) \to \sigma_i$ for some intuitionistic formulae σ_i, δ_i. For each such occurence, we construct a debate t_i for the corresponding Peirce's law as we did in the previous section. Then the term t^* obtained by applying the substitutions $[x_i \prec_{((\sigma_i \to \delta_i) \to \sigma_i) \to \sigma_i} t_i / x_i] t_0$ for all $i \leq n$ is a debate for T. As the t_i reduce to closed terms for Peirce's law and the x_i are wrapped in throw-steal-catch-terms after the substitutions, there exists a reduction (e.g. via call-by-onus) that throws away all x_i and reduces t^* to a closed term $t^*_{closed} : T$. □

6. Future Work and Conclusion

We have introduced a novel approach to represent arguments and debates as $\bar{\lambda}\mu\tilde{\mu}$ terms, furnishing argument and debate objects with computational content. Arguments are conceived as linear, potentially open $\bar{\lambda}\mu\tilde{\mu}$ terms and debates compose arguments via the support and attack syntactic patterns. In case of support, either the original argument or its supporter is returned, based on whether the supporter is considered redundant. As for attacks, a successful attack results in an exception that aborts the defeated arguments computation. The resulting *AC/DC* sublanguage is classical as it allows a debate for Peirce's law via a term that normalizes to a $\bar{\lambda}\mu\tilde{\mu}$ proof. An implementation of *AC/DC* based on the Fellowship prover [28] is under development.[2] In future work we will investigate open debates, i.e. debates outside the $\mathscr{S} - reduction - closed fragment$ in terms of Dung-style argumentation semantics. We will also present a fully general reduction strategy for *AC/DC* terms and extend the syntax of the language to include genuine enthymemes replacing the use of free variables. This will be the base for interactive construction of arguments and debates. Further research directions include correspondences between *AC/DC* debate terms and Lorenzen game winning strategies, a first order extension of the language and its use as a logical framework in the style of *Isabelle*.

[2] https://github.com/rappatoni/fellowship-mod

References

[1] Prakken H. An Abstract Framework for Argumentation with Structured Arguments. Argument & Computation. 2010;1(2):93-124.

[2] Modgil S, Prakken H. The ASPIC+ Framework for Structured Argumentation: A Tutorial. Argument & Computation. 2014;5(1):31-62.

[3] Bondarenko A, Dung PM, Kowalski RA, Toni F. An Abstract, Argumentation-Theoretic Approach to Default Reasoning. Artificial Intelligence. 1997;93(1–2):63-101.

[4] Gordon TF, Prakken H, Walton D. The Carneades Model of Argument and Burden of Proof. Artificial Intelligence. 2007;171(10–15):875-96.

[5] Besnard P, Hunter A. A Logic-Based Theory of Deductive Arguments. Artificial Intelligence. 2001;128(1–2):203-35.

[6] Arieli O, Straßer C. Sequent-Based Logical Argumentation. Argument & Computation. 2015;6(1):73-99.

[7] Harper R, Honsell F, Plotkin G. A Framework for Defining Logics. Journal of the ACM. 1993;40(1):143-84.

[8] Coquand T, Huet G. The Calculus of Constructions. Information and Computation. 1988;76(2–3):95-120.

[9] Church A. A Formulation of the Simple Theory of Types. The Journal of Symbolic Logic. 1940;5(2):56-68.

[10] Kohlhase M, Rabe F. A Scalable Module System. Information and Computation. 2013;230:1-54.

[11] Bertot Y, Castéran P. Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions. Texts in Theoretical Computer Science. Springer; 2004.

[12] de Moura L, Kong S, Avigad J, van Doorn F, von Raumer J. The Lean Theorem Prover. In: Automated Deduction – CADE-25. vol. 9195 of Lecture Notes in Computer Science. Springer; 2015. p. 378-88.

[13] Nipkow T, Paulson LC, Wenzel M. Isabelle/HOL: A Proof Assistant for Higher-Order Logic. vol. 2283 of Lecture Notes in Computer Science. Springer; 2002.

[14] Fuenmayor D, Benzmüller C. Computer-Supported Analysis of Arguments in Climate Engineering. In: Dastani M, Dong H, van der Torre L, editors. Logic and Argumentation. vol. 12061 of Lecture Notes in Computer Science. Cham: Springer; 2020. p. 104-15.

[15] Krause PJ, Ambler S, Elvang-Gøransson M, Fox J. A Logic of Argumentation for Reasoning under Uncertainty. Computational Intelligence. 1995;11(1):113-31.

[16] Ambler S. A Categorical Approach to the Semantics of Argumentation. Mathematical Structures in Computer Science. 1996;6(2):167-88.

[17] Pandžić S. A Logic of Defeasible Argumentation: Constructing Arguments in Justification Logic. Argument and Computation. 2022;13(1):3-47.

[18] Sørensen MH, Urzyczyn P. Lectures on the Curry-Howard Isomorphism. vol. 149 of Studies in Logic and the Foundations of Mathematics. Amsterdam: Elsevier; 2006.

[19] Griffin TG. A Formulae-as-Types Notion of Control. In: Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'90). ACM; 1990. p. 47-58.

[20] Parigot M. $\lambda\mu$-Calculus: An algorithmic interpretation of classical natural deduction. In: Voronkov A, editor. Logic Programming and Automated Reasoning. vol. 624 of Lecture Notes in Computer Science. Berlin/Heidelberg: Springer-Verlag; 1992. p. 190-201.

[21] Wadler P. Call-by-value is dual to call-by-name. In: Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming (ICFP '03). New York, NY, USA: Association for Computing Machinery; 2003. p. 189-201.

[22] Curien PL, Herbelin H. The Duality of Computation. In: Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00). New York, NY, USA: Association for Computing Machinery; 2000. p. 233-43.

[23] Herbelin H. C'est maintenant qu'on calcule: au cœur de la dualité [Habilitation thesis]. Université Paris 11; 2005.

[24] Lovas W, Crary K. Structural Normalization for Classical Natural Deduction; 2006. Manuscript, December 22, 2006.

[25] Harper R. Practical Foundations for Programming Languages. 2nd ed. Cambridge University Press; 2016.

[26] van Bakel S. Exception Handling and Classical Logic. In: Proceedings of the 21st International Symposium on Principles and Practice of Declarative Programming. PPDP 2019. ACM; 2019. p. 21:1-21:14.

[27] Autexier S, Sacerdoti-Coen C. A Formal Correspondence between OMDoc with Alternative Proofs and the $\lambda\mu\mu$-Calculus. In: Mathematical Knowledge Management. vol. 4108 of Lecture Notes in Computer Science. Berlin, Heidelberg: Springer; 2006. p. 67-81. Available from: https://doi.org/10.1007/11812289_7.

[28] Kirchner F. Systèmes de preuve interopérables. (Interoperable Proof Systems). Palaiseau, France: École Polytechnique; 2007. PhD thesis. Available from: https://pastel.archives-ouvertes.fr/pastel-00003192.